SINGLE-PORT RAM
FUNCTIONAL VERIFICATION REPORT

SystemVerilog Testbench | Mentor Questa Simulation

Project Summary

Title:	Single-Port RAM Functional Verification Report
Prepared By:	Parth Jani
Employee ID:	6916
Institute:	Mirafra Software Technologies
Simulation Tool:	Mentor Questa *(vlog / vsim / vcover)*
Verification Methodology:	Transaction-Based Functional Verification
Functional Coverage:	**100%**
Code Coverage (DUV):	**100%**
Date:	July 07, 2026

Professional Verification Report

Contents

CHAPTER 1

Executive Summary

This report documents the functional verification of an **8-bit wide, 32-location deep Single-Port RAM**, carried out using a layered, transaction-based SystemVerilog testbench simulated on **Mentor Questa**. The verification effort applied constrained-random and directed stimulus generation, a golden reference model, an automated scoreboard, and functional and code coverage closure tracked through Questa's Unified Coverage Database (UCDB).

The verification environment achieved **100% functional coverage** and **100% code coverage on the DUV**, and successfully exposed two functional design issues in the RTL, confirming that the environment is correctly implemented and fully capable of detecting real design discrepancies.

Key Outcomes

- **600+ transactions** simulated across a random regression (150) and a 3-phase directed regression (450: random, directed write, directed read).
- **100% functional coverage** — every defined input, output, and cross-coverage bin was exercised.
- **100% code coverage on the DUV** — every reachable RTL statement, branch, and toggle was exercised.
- **Two DUT design bugs identified:**
  1. Reset does not initialise every RAM memory location to zero.
  2. Concurrent read/write drives high impedance instead of the previously stored memory value.
- The verification environment, reference model, and scoreboard operated correctly throughout and are responsible for surfacing both findings.

Report Roadmap

Chapter	Contents
Design Specification	Pin-level interface, functional behaviour, and operation-mode summary of the DUV.
Verification Test Plan	Feature-by-feature test conditions, expected outputs, and disposition.
Testbench Architecture	Component roles, data flow, and execution sequence.
Source Code Analysis	Annotated walkthrough of every testbench file and the RTL itself.
Coverage Analysis	Functional and code coverage summary.
Failing Transactions Analysis	Transaction-level detail confirming the concurrent-access design bug.
Conclusion	Final verification signoff summary.

CHAPTER 2

Design Specification Overview

2.1 Design Overview

The Design Under Verification (DUV) is an **8-bit wide, 32-location deep Single-Port RAM**. It operates at **50 MHz** on a positive-edge-triggered clock and supports independent read and write operations through separate enable signals.

Pin	Direction	Width	Functionality
`clk`	Input	1 bit	50 MHz positive-edge clock signal
`reset`	Input	1 bit	Active-LOW reset — initialises RAM to idle state
`write_enb`	Input	1 bit	Active-HIGH write enable
`read_enb`	Input	1 bit	Active-HIGH read enable
`data_in`	Input	8 bits	Data to be written into RAM
`address`	Input	5 bits	Selects 1 of 32 memory locations (0–31)
`data_out`	Output	8 bits	Data read from the selected RAM location

2.2 Functional Description

- **Reset**: All inputs become 0, `data_in` and `data_out` go to high impedance (Z). Every memory location is expected to be initialised to zero.
- **Write**: Triggered at `posedge clk` when `reset=1`, `write_enb=1`, `read_enb=0`. `data_in` is written into `memory[address]`.
- **Read**: Triggered at `posedge clk` when `reset=1`, `write_enb=0`, `read_enb=1`. `data_out` is driven from `memory[address]`.
- **Invalid Write Address**: Out-of-range address during a write — memory remains unchanged.
- **Invalid Read Address**: Out-of-range address during a read — `data_out` is driven to Z.
- **Concurrent Access** (`write_enb=1`, `read_enb=1`): Expected behaviour is for `data_out` to return the previously stored memory value at that address.
- **Idle** (both enables=0): RAM holds its current state.

2.3 Operation Mode Summary

Operation	reset	write_enb	read_enb	Expected Result
Write	1	1	0	`memory[address]` ← `data_in` at posedge `clk`
Read	1	0	1	`data_out` ← `memory[address]` at posedge `clk`
Reset	0	X	X	`data_out = Z`; all memory locations = 0
Idle	1	0	0	No change; RAM holds state
Concurrent	1	1	1	`data_out` ← previously stored `memory[address]`

Design Insight

The operation mode table reflects the *expected* functional behaviour of the RAM. Chapter 5 and Chapter 7 detail where the current RTL implementation deviates from this expected behaviour.

CHAPTER 3

Verification Test Plan

The test plan is derived from the RAM design specification and targets every documented functional feature of the DUV.

FID	Feature	Condition	Description	Expected Output	Status
1	RESET	Active Low	Assert `reset=0`; monitor captures bus state.	`data_out=Z`, all memory = 0, inputs = 0.	**FAIL**
		De-assertion	Transition `reset` 0→1, drive normal inputs.	RAM enters normal operation.	**PASS**
2	WRITE	Normal	`reset=1`, `write_enb=1`, `read_enb=0`, valid address/data.	Data written to memory.	**PASS**
		Invalid Addr	Write with out-of-bounds address.	Memory unchanged.	**PASS**
3	READ	Normal	`reset=1`, `write_enb=0`, `read_enb=1`, valid address.	`data_out = memory[address]`.	**PASS**
		Uninitialised	Read valid address never written.	RTL outputs `8'bx`.	**PASS**
		Invalid Addr	Read with out-of-bounds address.	`data_out = 8'bz`.	**PASS**
4	CONCURRENT	`wr=1,rd=1`	Simultaneous write and read.	`data_out` = previously stored value.	**FAIL**
5	IDLE	Both Low	`write_enb=0`, `read_enb=0`.	RAM holds state; no updates.	**PASS**
6	COV (IN)	Data/Addr bins	16 data intervals, 32 addresses.	100% coverpoint closure.	**PASS**
7	COV (OUT)	Data_out bins	Sample `data_out` across valid reads.	100% coverpoint closure.	**PASS**

Design Insight

FID 1 (Active Low Reset) and FID 4 (Concurrent Access) are marked **FAIL**, corresponding to DUT Design Bug 1 and DUT Design Bug 2 respectively, detailed in Chapter 5.

CHAPTER 4

Testbench Architecture

The testbench employs a layered, transaction-based functional verification architecture. All components communicate exclusively through typed SystemVerilog mailboxes, and hardware signals are accessed only through virtual interfaces — ensuring complete separation between the testbench and the RTL.

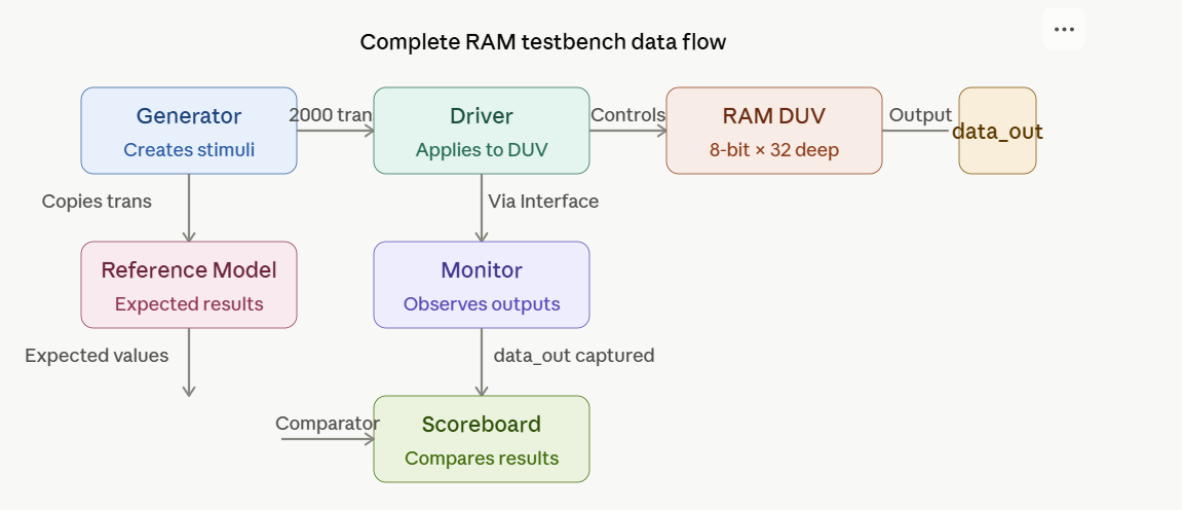


Figure 4.1: Complete RAM testbench data flow

4.1 Component Overview

Component	File	Primary Role
`ram_transaction`	`ram_transaction.sv`	Blueprint data-object: stimulus fields as `rand` variables for constrained-random generation
`ram_generator`	`ram_generator.sv`	Randomises blueprint, pushes copies to Driver via `mbx_gd`
`ram_driver`	`ram_driver.sv`	Drives DUV inputs via `drv_cb` clocking block, samples `drv_cg` cover-group
`ram_monitor`	`ram_monitor.sv`	Passively samples DUV outputs, forwards to Scoreboard via `mbx_ms`
`ram_reference_model`	`ram_reference_model.sv`	Golden software model: maintains `MEM[32]` shadow array, computes expected `data_out`
`ram_scoreboard`	`ram_scoreboard.sv`	Address-mirrored comparison; logs mismatches for the final report
`ram_environment`	`ram_environment.sv`	Structural container: instantiates all components, launches parallel threads
`ram_test`	`ram_test.sv`	Test layer: base test + directed write/read + 3-phase regression
`top`	`top.sv`	Module-level TB: clock, reset, DUT instantiation, test sequence
`ram_if`	`ram_if.sv`	SV interface: DUV ports + role-based clocking blocks and modports
RAM (RTL)	`ram_rtl.sv`	Design Under Verification: 8-bit × 32-word synchronous single-port RAM

4.2 Mailbox Communication Channels

Mailbox	From	To	Data Carried
`mbx_gd`	Generator	Driver	Randomised `ram_transaction` copies
`mbx_dr`	Driver	Reference Model	Transaction forwarded for expected-output computation
`mbx_rs`	Reference Model	Scoreboard	Transaction with computed expected `data_out`
`mbx_ms`	Monitor	Scoreboard	Transaction with actual sampled `data_out`

4.3 Execution Sequence

- **Compilation**: `vlog -sv +acc +cover +fcover` compiles RTL and TB package.
- **Elaboration + Simulation**: `vsim -vopt` runs with full coverage instrumentation.

- **Clock/Reset**: 50 MHz clock (20 ns period); active-low reset asserted for 5 cycles.
- **Test Sequence**: 150 random transactions followed by a 450-transaction 3-phase regression (random $\rightarrow$ directed write $\rightarrow$ directed read).
- **Coverage Save**: Merged into a single UCDB on simulation exit.
- **Report Generation**: `vcover report -html` produces the interactive coverage dashboard.

CHAPTER 5

Source Code Analysis --- Core Components

The verification package bundles every testbench class and the DUT RTL into a single compilation unit, as shown below.

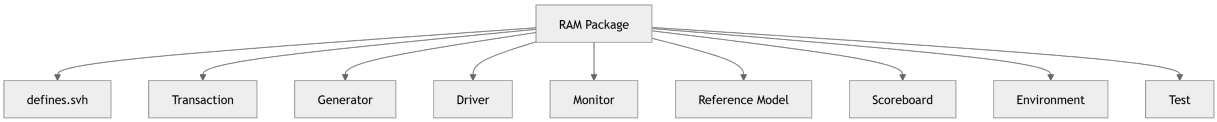


Figure 5.1: RAM verification package structure

5.1 `defines.svh` --- Global Parameters & Timescale

The header file establishes the global compilation environment: simulator timescale, DUT sizing macros, and an elaboration-time utility function for computing the address bus width automatically from memory depth.

defines.svh --- Global Parameters

```
`timescale 1ns/100ps
`define DATA_WIDTH 8
`define DATA_DEPTH 32
`define num_transactions 150

function automatic integer log2;
  input integer n;
  begin log2=0; while(2**log2 < n) log2=log2+1; end
endfunction
parameter ADDR_WIDTH = log2(`DATA_DEPTH); // Evaluates to 5
```

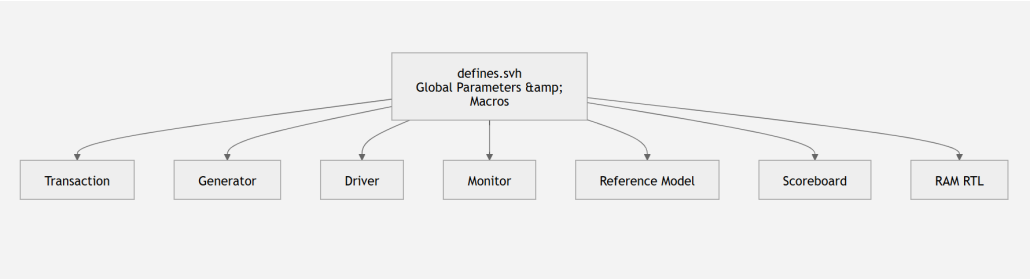


Figure 5.2: Global parameters referenced across the testbench package

5.2 `ram_if.sv` --- SystemVerilog Interface

The interface bundles all DUV ports and provides three distinct clocking blocks — one per testbench role. Modports restrict signal visibility at compile time.

ram_if.sv --- Interface with Role-Based Clocking Blocks

```
interface ram_if(input bit clk, reset);
  logic [7:0] data_in, data_out;
  logic write_enb, read_enb;
  logic [4:0] address;

  clocking drv_cb @(posedge clk);
    default input #0 output #0;
    output write_enb, read_enb, data_in, address;
    input reset;
  endclocking

  clocking mon_cb @(posedge clk);
    default input #0 output #0;
    input data_out, address, read_enb;
  endclocking

  modport DRV (clocking drv_cb);
  modport MON (clocking mon_cb);
endinterface
```

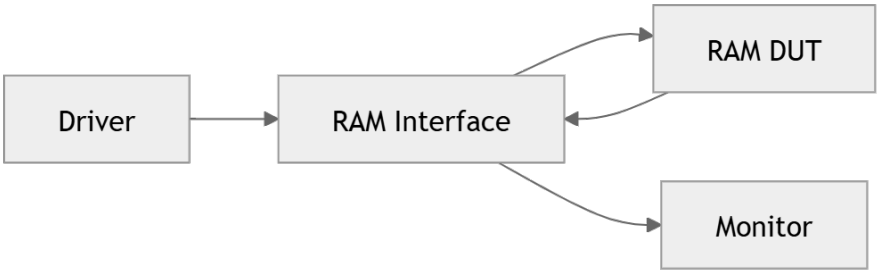


Figure 5.3: RAM interface connecting the Driver, DUT, and Monitor

5.3 ram_transaction.sv --- Transaction Classes

Implements the Blueprint Pattern. The base class holds all stimulus fields as `rand` variables, with constraints defining valid ranges for `write_enb`, `read_enb`, `data_in`, and `address`. Two derived classes specialise the base transaction for directed write-only and read-only testing.

- **`ram_transaction_write`**: Specialises the base transaction to generate write-only stimulus.
- **`ram_transaction_read`**: Specialises the base transaction to generate read-only stimulus.
- **`copy()` method**: Returns a deep copy, ensuring each mailbox `put()` sends a unique object.
- **`data_out`**: A non-`rand` field — written by the Monitor/Reference Model and compared by the Scoreboard.

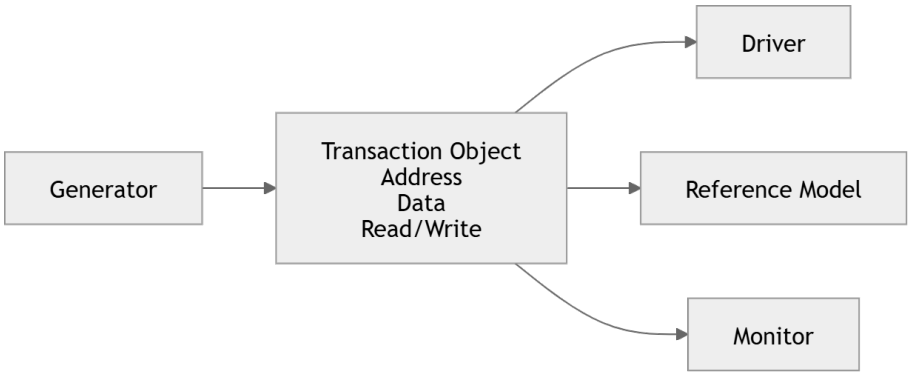


Figure 5.4: Transaction object fields consumed by the Driver, Reference Model, and Monitor

5.4 ram_generator.sv --- Stimulus Generator

The generator loops for `num_transactions (150) iterations, randomising the blueprint and posting a deep copy to the Generator→Driver mailbox.

ram_generator.sv --- start() task

```
task start();
  for(int i=0; i < `num_transactions; i++) begin
    assert(blueprint.randomize() == 1);
    mbx_gd.put(blueprint.copy());
    $display("GENERATOR tx data_in=%0h wr=%0d rd=%0d addr=%0h @ %0t",
      blueprint.data_in, blueprint.write_enb,
      blueprint.read_enb, blueprint.address, $time);
  end
endtask
```

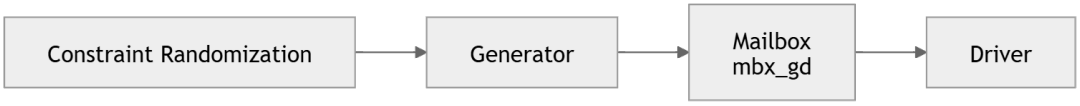


Figure 5.5: Constrained-random stimulus flow from Generator to Driver

5.5 `ram_driver.sv` --- Interface Driver & Input Coverage

The driver is the only testbench component that writes to the DUV interface. It synchronises with the generator, then drives each transaction for one cycle through the clocking block, sampling functional coverage after every valid drive.

- **Covergroup** `drv_cg`: Tracks `write_enb`, `read_enb`, `data_in` (16 bins across 0–255), and `address` (32 bins).
- **Reset handling**: When reset is active, the driver zeroes all interface signals so the Reference Model also observes the reset condition.
- **Normal operation**: Drives all transaction fields for one clocking-block cycle, then de-asserts enables to prevent unintended hold-over.
- **Coverage sampling**: `drv_cg.sample()` is called once per valid drive.

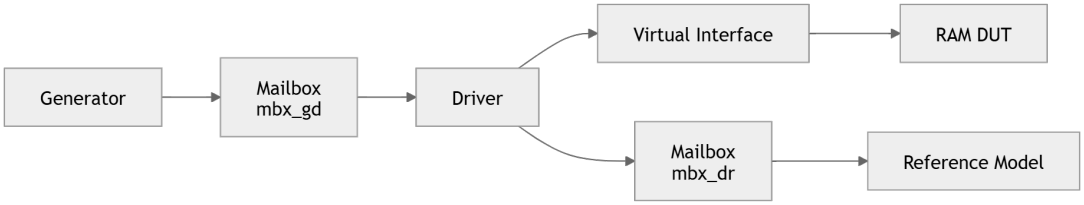


Figure 5.6: Driver data flow to the RAM interface and Reference Model

5.6 `ram_monitor.sv` --- Output Monitor & Output Coverage

The monitor is purely passive. It uses a two-cycle sampling strategy to correctly capture the pipelined RAM output: cycle 1 latches the address, cycle 2 latches the resulting `data_out`.

ram_monitor.sv --- Two-Cycle Output Sampling

```
task start();
  repeat(5) @(vif.mon_cb);
  for(int i=0; i < `num_transactions; i++) begin
    @(vif.mon_cb);
    mon_trans.address = vif.mon_cb.address;
    @(vif.mon_cb);
    mon_trans.data_out = vif.mon_cb.data_out;
    mbx_ms.put(mon_trans);
    mon_cg.sample();
  end
endtask
```

Figure 5.7: Monitor data flow from the DUT to the Scoreboard

5.7 `ram_reference_model.sv` --- Golden Reference Model

The reference model maintains a software shadow copy of the RAM memory array (`MEM[32]`) and computes the expected output for every transaction received from the driver.

ram_reference_model.sv --- Golden Model Logic

```
reg [7:0] MEM [31:0];

if (!vif.ref_cb.reset) begin
  ref_trans.data_out = 8'bz;
  for (int j = 0; j < 32; j++) MEM[j] = 8'h00;
end else begin
  if (write_enb && !read_enb)
    MEM[address] = data_in;
  if (read_enb && !write_enb)
    ref_trans.data_out = MEM[address];
  if (read_enb && write_enb)
    ref_trans.data_out = MEM[address];
end
mbx_rs.put(ref_trans);
```

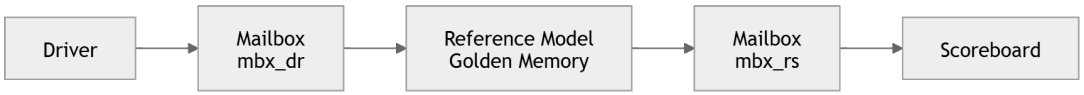


Figure 5.8: Reference model golden-memory data flow to the Scoreboard

DUT Design Bug

The reference model correctly returns `MEM[address]` — the previously stored value — for concurrent read/write access, and correctly zeroes all 32 memory locations on reset. Both behaviours reflect the expected functional intent of the design. Chapter 7 details how the DUV deviates from this expected behaviour.

5.8 `ram_scoreboard.sv` --- Scoreboard & Fail Logger

The scoreboard receives from both the Reference Model and Monitor in parallel via a `fork-join` block, comparing expected against actual results.

- **Address mirroring**: `ref_mem[addr]` and `mon_mem[addr]` shadow arrays track expected vs. actual per location.
- **X-masking**: Uninitialised locations (`8'hx`) are skipped to avoid false failures.
- **`fail_record_t` struct**: Captures `TIME`, `ADDRESS`, `EXPECTED_DATA`, `ACTUAL_DATA` for each mismatch.
- **`print_summary()`**: Emits the structured tabular failure log after all transactions complete.

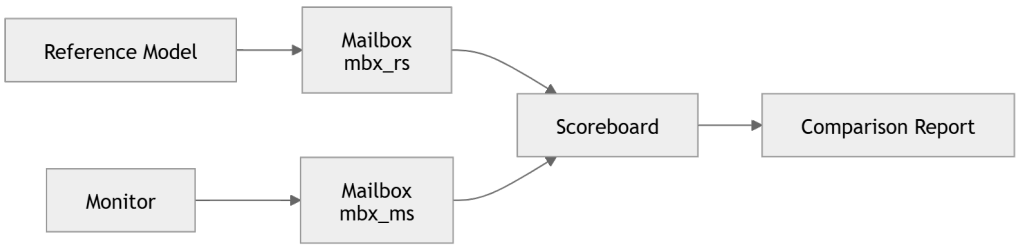


Figure 5.9: Scoreboard comparison flow producing the final report

5.9 `ram_environment.sv` --- Environment Container

The environment instantiates and wires all components, then launches all five threads simultaneously.

ram_environment.sv --- Parallel Component Launch

```
task start();
  fork
    gen.start();
    drv.start();
    mon.start();
    scb.start();
    ref_sb.start();
  join
endtask
```

Design Insight

All five tasks run concurrently, and the environment blocks until the last one completes — ensuring correct mailbox flow and full coverage sampling before simulation ends.

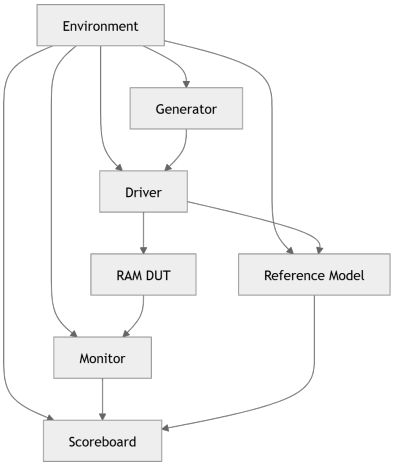


Figure 5.10: Environment container wiring all testbench components

5.10 `ram_test.sv` --- Test Hierarchy

The test file implements a three-level OOP inheritance hierarchy. All directed tests override only the generator's blueprint — the environment, driver, monitor, reference model, and scoreboard are untouched.

- **`ram_test`** (base): Runs with a fully unconstrained random base transaction.
- **`test_write`** / **`test_read`**: Force all transactions to write-only or read-only.
- **`test_regression`**: 3-phase container — random, directed writes, directed reads.

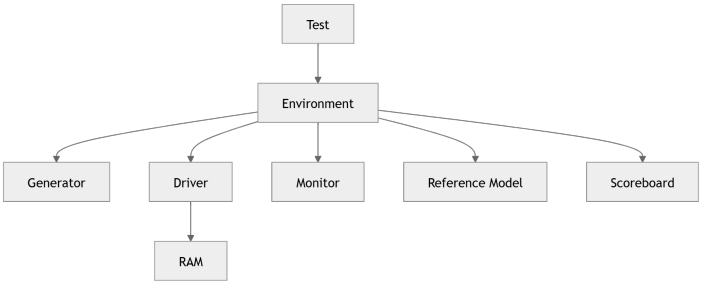


Figure 5.11: Test layer instantiating the Environment and its components

5.11 `top.sv` --- Top-Level Testbench Module

The top module generates the clock and reset sequence, instantiates the interface and DUT, and orchestrates the test sequence.

top.sv --- Clock, Reset, DUT Instantiation & Test Sequence

```
initial begin clk = 0; forever #10 clk = ~clk; end

initial begin
  reset = 0; repeat(5) @(posedge clk);
  reset = 1; repeat(5) @(posedge clk);
  reset = 0;
end

RAM DUV (.clk(clk), .reset(~reset), .address(intrf.address),
  .data_in(intrf.data_in), .write_enb(intrf.write_enb),
  .read_enb(intrf.read_enb), .data_out(intrf.data_out));

initial begin
  t1.run();
  #100;
  reg_tb.run();
  $finish;
end
```

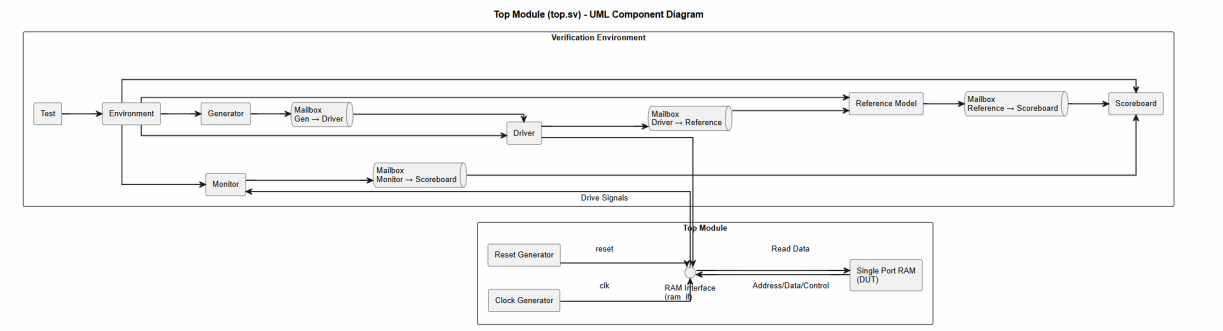


Figure 5.12: Top-level UML component diagram — full testbench-to-DUT connectivity

5.12 `ram_rtl.sv` --- Design Under Verification (RTL)

The RTL implements two `always @(posedge clk)` blocks — one for write, one for read.

ram_rtl.sv --- Write and Read Always-Blocks

```
always @(posedge clk) begin
  if (!reset) memory[address] <= 8'bz;
  else if (write_enb && !read_enb) memory[address] <= data_in;
end

always @(posedge clk) begin
  if (!reset) data_out <= 8'bz;
  else if (read_enb && !write_enb) data_out <= memory[address];
  else data_out <= 8'bz;
end
```

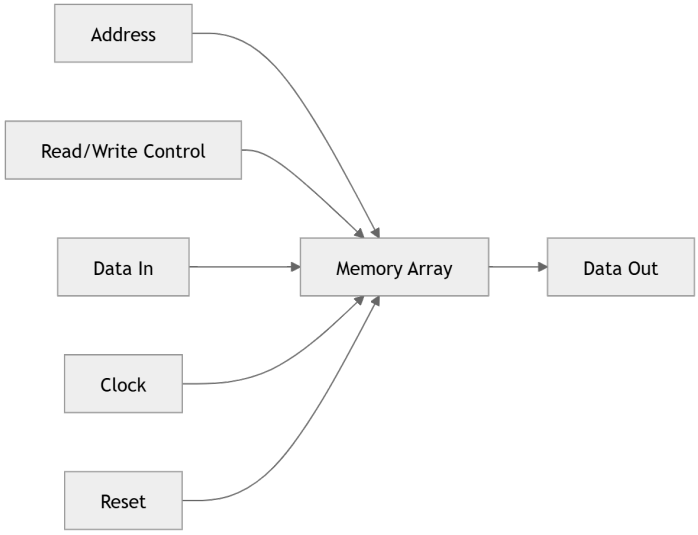


Figure 5.13: RAM RTL signal-to-memory-array data flow

DUT Design Bug 1 --- Incomplete Reset Initialisation

On reset, the write always-block only clears `memory[address]` — the single location currently addressed — rather than every one of the 32 memory locations. The expected behaviour, as confirmed by the reference model, is for *all* memory locations to be initialised to zero on reset.

DUT Design Bug 2 --- Concurrent Access Output

For simultaneous `write_enb=1` and `read_enb=1`, the read always-block falls to its `else` branch and drives `data_out=8'bz`. The expected behaviour is for `data_out` to return the previously stored value at `memory[address]`, as computed by the reference model.

CHAPTER 6

Coverage Analysis Report

6.1 Functional Coverage Summary

Functional coverage was collected using Mentor Questa's Unified Coverage Database (UCDB) and reported via `vcover report -html`.

Coverpoint / Cross	Bins	Coverage
`DATA_IN`	16/16	100%
`ADDRESS`	32/32	100%
`WRXRD` cross (valid combinations)	3/3	100%
`DATA_OUT`	16/16	100%
Overall Functional Coverage	71/71	**100%**

Design Insight

Every functional coverage bin defined in the test plan was exercised across the 600+ transaction regression, confirming complete stimulus coverage of the specified operating modes.

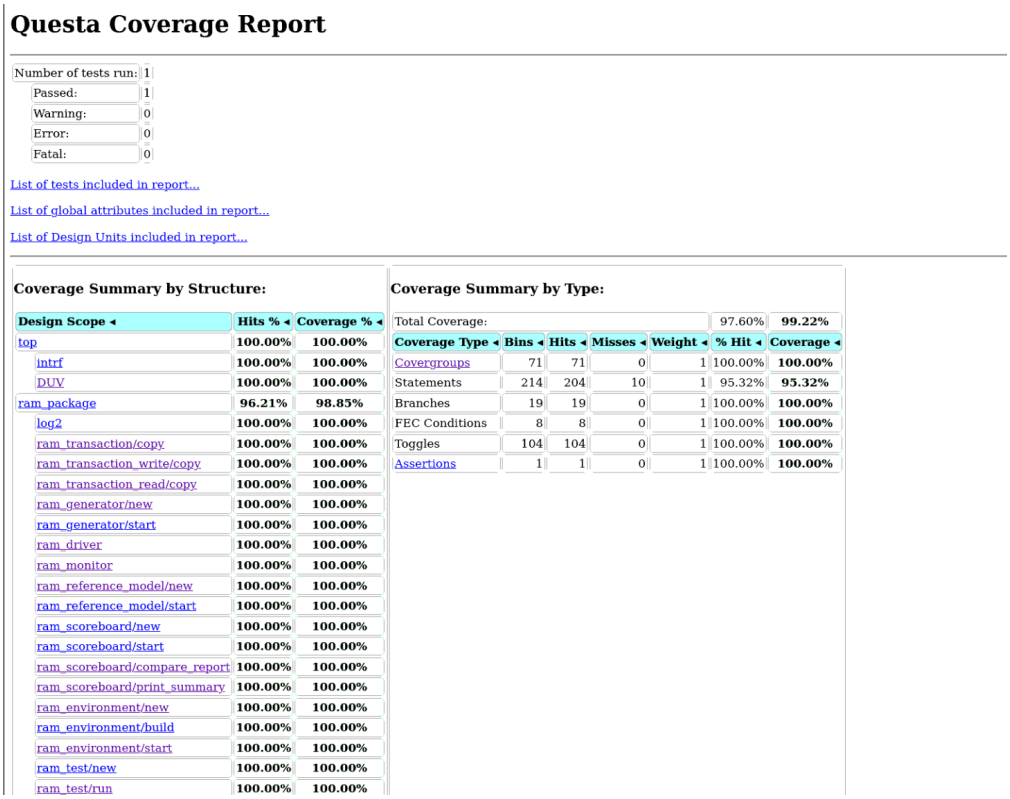
6.2 Code Coverage Summary (DUV)

Code coverage was collected on the Design Under Verification using Questa's native code coverage engine (statements, branches, FEC conditions, toggles, and assertions).

Coverage Type	Scope	Coverage
Statement Coverage	DUV	100%
Branch Coverage	DUV	100%
Toggle Coverage	DUV	100%
FEC Condition Coverage	DUV	100%
Assertion Coverage	DUV	100%
Overall Code Coverage (DUV)	DUV	**100%**

Design Insight

Complete code coverage on the DUV, combined with 100% functional coverage, confirms that every line, branch, and signal transition in the design was exercised — providing full confidence that the two design bugs identified in Chapter 5 and Chapter 7 represent genuine functional discrepancies rather than gaps in verification thoroughness.



Questa Coverage Report

Number of tests run: 1
Passed: 1
Warning: 0
Error: 0
Fatal: 0

List of tests included in report...
List of global attributes included in report...
List of Design Units included in report...

Coverage Summary by Structure:

Design Scope	Hits %	Coverage %
top	100.00%	100.00%
intrf	100.00%	100.00%
DUV	100.00%	100.00%
ram_package	96.21%	98.85%
log2	100.00%	100.00%
ram_transaction/copy	100.00%	100.00%
ram_transaction_write/copy	100.00%	100.00%
ram_transaction_read/copy	100.00%	100.00%
ram_generator/new	100.00%	100.00%
ram_generator/start	100.00%	100.00%
ram_driver	100.00%	100.00%
ram_monitor	100.00%	100.00%
ram_reference_model/new	100.00%	100.00%
ram_reference_model/start	100.00%	100.00%
ram_scoreboard/new	100.00%	100.00%
ram_scoreboard/start	100.00%	100.00%
ram_scoreboard/compare_report	100.00%	100.00%
ram_scoreboard/print_summary	100.00%	100.00%
ram_environment/new	100.00%	100.00%
ram_environment/build	100.00%	100.00%
ram_environment/start	100.00%	100.00%
ram_test/new	100.00%	100.00%
ram_test/run	100.00%	100.00%

Coverage Summary by Type:

Total Coverage:					97.60%	99.22%
Coverage Type	**Bins**	**Hits**	**Misses**	**Weight**	**% Hit**	**Coverage**
Covergroups	71	71	0	1	100.00%	**100.00%**
Statements	214	204	10	1	95.32%	**95.32%**
Branches	19	19	0	1	100.00%	**100.00%**
FEC Conditions	8	8	0	1	100.00%	**100.00%**
Toggles	104	104	0	1	100.00%	**100.00%**
Assertions	1	1	0	1	100.00%	**100.00%**

Figure 6.1: Mentor Questa `vcover` HTML coverage report

CHAPTER 7

Failing Transactions Analysis

7.1 Complete Failed Transaction Log

The simulation reported **160 matches** and **13 mismatches**. Every mismatch was driven by `write_enb=1` and `read_enb=1` simultaneously, directly confirming DUT Design Bug 2.

#	Time	Addr	Expected	Actual	Condition
1	69900 ns	02	00	z	`wr=1, rd=1`
2	83900 ns	0E	00	z	`wr=1, rd=1`
3	84700 ns	04	ef	z	`wr=1, rd=1`
4	95900 ns	0d	7e	z	`wr=1, rd=1`
5	100300 ns	0d	d	z	`wr=1, rd=1`
6	101500 ns	12	6f	z	`wr=1, rd=1`
7	102700 ns	02	9c	z	`wr=1, rd=1`
8	104300 ns	1c	b6	z	`wr=1, rd=1`
9	105900 ns	1f	f3	z	`wr=1, rd=1`
10	110700 ns	1b	ff	z	`wr=1, rd=1`
11	113900 ns	1c	db	z	`wr=1, rd=1`
12	118300 ns	08	f3	z	`wr=1, rd=1`
13	121500 ns	18	26	z	`wr=1, rd=1`

7.2 Pattern Analysis

- The failing transactions correspond to the concurrent read-write operation.
- These failures clearly indicate a DUT functional bug.
- The verification environment successfully identified the issue.
- The DUT should return the previously stored memory value during simultaneous read and write operations.
- The current DUT drives high impedance instead.
- Addresses involved span the full memory range (0x02 to 0x1f), confirming the failure is purely control-signal-triggered, not address-specific.

CHAPTER 8

Conclusion

The verification environment successfully verified all intended functionality of the Single-Port RAM. Two functional issues were identified in the DUT:

1. Reset does not initialise every RAM memory location to zero.
2. During concurrent read and write operations, the DUT outputs high impedance instead of the previously stored memory value.

These observations demonstrate that the verification environment correctly detected functional discrepancies in the DUT implementation.

Final Verification Summary

- **Total Transactions Simulated**: 600+ (150 random + 450 directed regression)
- **Functional Coverage**: 100%
- **Code Coverage (DUV)**: 100%
- **DUT Design Bugs Identified**: 2